

# FightDFear

## Project Structure & Technical Blueprint

Live System Audit · Tech Stack · Database Design · Module Breakdown · Roadmap

**Spring Boot 3.4.3**

Backend framework

**Java 17**

Language

**MySQL 8**

Primary DB

**JSP / JSTL**

Web frontend

**STOMP  
WebSocket**

Real-time

**Weka 3.8.6**

AI safe routes

This blueprint is generated directly from the current FightDFear Spring Boot codebase (controllers, entities, repositories, and configuration as of this audit) plus the recommended additions discussed for investor-ready hardening and the mobile safety build. Every item is marked **Live** (already built and working today) or **Planned** (recommended next, not yet built) so this document can be shared with investors or engineers without overstating what exists.

### SECTION 1 — CURRENT & RECOMMENDED TECH STACK

#### Stack Status — What’s Actually Running Today

| Layer                 | Technology                                  | Status  | Why                                                                                                              |
|-----------------------|---------------------------------------------|---------|------------------------------------------------------------------------------------------------------------------|
| Backend API           | Spring Boot 3.4.3 (Java 17)                 | LIVE    | Mature enterprise stack, built-in Spring Security, JPA/Hibernate ORM. Team already knows it — no rewrite needed. |
| Web frontend          | JSP / JSTL (server-side rendered)           | LIVE    | 215 JSP views rendering directly from the backend. Functional, but dated — no SPA, no offline capability.        |
| Mobile apps           | React Native or Flutter                     | PLANNED | Required for background location, hardware-button SOS, and lock-screen alerts — things a browser tab cannot do.  |
| Primary database      | MySQL 8                                     | LIVE    | womenbesafe schema, ~85 JPA entities. Single instance, no read replica.                                          |
| ORM                   | Spring Data JPA / Hibernate                 | LIVE    | 75 repositories. Schema currently auto-updates (ddl-auto=update) rather than versioned migrations.               |
| Real-time channel     | Spring WebSocket + STOMP (in-memory broker) | LIVE    | Relays SOS and chat at /ws-sos and /ws-chat. In-memory broker caps horizontal scaling.                           |
| Real-time (scaled)    | Redis Pub/Sub                               | PLANNED | Needed once concurrent WebSocket sessions outgrow a single server instance.                                      |
| AI / ML               | Weka 3.8.6 (embedded, in-JVM)               | LIVE    | Powers AI-suggested safe routes using historical crime/danger-point data.                                        |
| Payments              | Razorpay Java SDK                           | LIVE    | Test-mode keys currently in source — flagged in Section 8.                                                       |
| Email                 | JavaMail via Gmail SMTP                     | LIVE    | Trusted-contact alert emails, password reset. Single mailbox, no dedicated transactional provider.               |
| File storage          | Local disk (webapp/uploads)                 | LIVE    | Profile photos, videos, incident media stored on the app server itself.                                          |
| File storage (scaled) | AWS S3 / Cloudflare R2                      | PLANNED | Removes storage from the app server, enables CDN delivery for video/image content.                               |

| Layer               | Technology                    | Status  | Why                                                                                    |
|---------------------|-------------------------------|---------|----------------------------------------------------------------------------------------|
| PDF generation      | iText 5.5.13                  | LIVE    | Certificates and exportable documents.                                                 |
| Push notifications  | Firebase Cloud Messaging      | PLANNED | Required for mobile SOS/alert delivery — email/SMS alone is too slow for an emergency. |
| Deployment          | Manual / embedded Tomcat      | LIVE    | No Dockerfile or CI/CD pipeline currently in the repo.                                 |
| Deployment (scaled) | Docker + GitHub Actions CI/CD | PLANNED | Consistent environments, automated builds, faster and safer deploys.                   |
| Error tracking      | Sentry                        | PLANNED | No monitoring currently in place — failures are only visible in server logs.           |

## SECTION 2 — SYSTEM ARCHITECTURE OVERVIEW

**Architecture style:** single Spring Boot monolith serving server-rendered JSP views today, with a REST/WebSocket surface already in place that a future mobile app can consume directly — no backend rewrite required to add mobile.

| Layer         | Component                             | Responsibility                                                     | Port / Protocol | Status  |
|---------------|---------------------------------------|--------------------------------------------------------------------|-----------------|---------|
| Client        | Browser (JSP views)                   | Customer + admin web UI, current primary interface                 | 8084 / HTTP     | LIVE    |
| Client        | Mobile app (React Native / Flutter)   | Background SOS, live location, push alerts                         | Android / iOS   | PLANNED |
| API           | Spring MVC controllers                | 59 controllers — all business logic, auth, SOS, bookings, payments | 8084 / HTTP     | LIVE    |
| Real-time     | Spring WebSocket (STOMP, SockJS over) | Live SOS broadcast, chat — in-memory, single instance              | 8084 / WS       | LIVE    |
| Service layer | 35 Spring services                    | Business logic, including embedded Weka for ML                     | 8084 / HTTP     | LIVE    |
| Database      | MySQL 8                               | All persistent data — users, SOS, bookings, marketplace, medical   | 3306            | LIVE    |
| Storage       | Local filesystem (uploads/)           | Profile photos, incident media, training videos                    | filesystem      | LIVE    |
| Payments      | Razorpay SDK                          | Payment order creation and verification                            | :443 / HTTPS    | LIVE    |
| Email         | Gmail SMTP                            | Trusted-contact alerts, password reset links                       | 587             | LIVE    |

SECTION 3 — PROJECT FOLDER STRUCTURE

Actual repository layout (Maven standard layout, single module):

```
FightDFear/                                <- Root Maven project
+-- pom.xml                                <- Spring Boot 3.4.3, Java 17
+-- src/main/java/in/sp/main/
|   +-- Controller/                        <- 59 controllers (MVC + REST)
|   +-- Service/                          <- ~35 services (business logic + AISafetyService)
|   +-- Repository/                      <- 75 Spring Data JPA repositories
|   +-- Entities/                        <- ~85 JPA entities
|   +-- Config/                          <- SecurityConfig, WebSocketConfig, SchedulingConfig
|   +-- dto/                             <- Request/response DTOs
+-- src/main/resources/
|   +-- application.properties            <- DB, mail, Razorpay, WebSocket config
|   +-- static/                          <- assets, beauty module static files
+-- src/main/webapp/
|   +-- WEB-INF/views/                   <- 215 JSP views
|   |   +-- doctor/ marketplace/ salon/ stylist/ user/ women-products/
|   |   +-- livelocation/ safeRoute/ questions/ profilechat/ fragments/
|   +-- resources/                       <- css/ js/ bootstrap/ images/
|   +-- uploads/                         <- user-uploaded media (local disk)
```

**Not present in the repo:** no Dockerfile, no docker-compose.yml, no .github/workflows/ — there is currently no containerization or CI/CD pipeline. See Section 8.

SECTION 4 — DATABASE DESIGN (MYSQL)

Core Tables — Actual Entity Fields

~85 JPA entities exist in total. The tables below are the ones that carry the product's core logic; fields are taken directly from the entity classes.

| Table                 | Key fields (actual)                                                                                               | Purpose                                          |
|-----------------------|-------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| users                 | id, fullName, phoneNumber, email, homeAddress, dob, age, identityDocument, verificationStatus,                    | Core identity document for verification          |
| trusted_contact       | id, name, phone, email, relation, isPrimary, canReceiveSMS/Email/Call, isTrusted                                  | SMS/Email/Call during an SOS                     |
| sos_request           | id, user, latitude, longitude, googleMapsLink, status, actedAt, respondedAt, escalatedAt, escalatedBy             | Status of SOS request, escalated with escalation |
| sos_alert             | id, userId, userName, userPhone, latitude, longitude, message/pinned/SOS alert, status, eventAt                   | Message/pinned SOS alert, status, eventAt        |
| journey_session       | id, user, destinationText, startLat/Lng, startFromText, createdAt, expectedArrival, status, alerted               | Travel with expected arrival, status, alerted    |
| medical_details       | id, bloodGroup, allergies, medicalHistory, medications, sensitive                                                 | Sensitive — flagged for at-rest encryption       |
| incident              | id, type, location, description, mediaPath, latitude, longitude, reportedBy, reportedAt, status                   | User-reported safety incidents with media        |
| booking               | id, user, salon, stylist, bookingTime, status, rating, pricePaid, paymentMode, razorpayOrderId, razorpayPaymentId | Salon/stylist booking with payment               |
| doctor_appointment    | id, user, doctor, appointmentTime, status, consultationType, healthType, doctorConsultations, amount              | Type, healthType, doctor consultations, amount   |
| women_product         | id, user, product, seller, quantity, totalPrice, paymentMode, shippingAddress, SOS rating, live                   | Track sales, shipping address, SOS rating, live  |
| online_class_enroll   | id, name, onlineClass, status                                                                                     | Self-defense class enrollment                    |
| training_progress     | id, moduleName, progressPercentage, jobAssistanceRequested, finalBinary, actionStack                              | Requested final binary, actionStack              |
| volunteer_sos_request | id, sosRequest, volunteer, status, respondedAt, message                                                           | Volunteer responder actions against a live       |
| badge                 | id, name, description, awardedAt, user                                                                            | Gamification layer — exists in schema, wo        |

Production-Readiness Gaps in the Current Database Layer

| Area                   | Current state                                                 | Recommended fix                                                                           |  |
|------------------------|---------------------------------------------------------------|-------------------------------------------------------------------------------------------|--|
| Schema versioning      | ddl-auto=update (auto-migrates on boot)                       | Move to Flyway or Liquibase with versioned, reviewable migration scripts                  |  |
| Connection handling    | Default Hibernate connection pool, no pooling                 | Add HikariCP tuning now; PgBouncer-equivalent (ProxySQL) only needed later                |  |
| Read scaling           | Single MySQL instance handles all reads and writes            | Add read replica once reporting/search queries start competing with OLTP                  |  |
| Sensitive data at rest | MedicalDetails and trustedContact fields stored in plain text | Encrypt with AES-GCM encryption via a JPA AttributeConverter — cover all sensitive fields |  |
| Backups                | No automated backup strategy visible in the code              | Scripted MySQL dumps to S3/R2 with a defined retention window                             |  |

## SECTION 5 — COMPLETE MODULE BREAKDOWN (LIVE ENDPOINTS)

Every endpoint below exists in the current codebase today — taken directly from controller annotations, not aspirational.

### Auth & account

|                                |                               |
|--------------------------------|-------------------------------|
| POST /login                    | - Session-based login         |
| GET/POST /auth/forgot-password | - Password reset request      |
| GET/POST /auth/reset-password  | - Password reset confirmation |
| GET/POST /users/register       | - New user registration       |
| GET /users/dashboard           | - User home dashboard         |

### SOS & emergency response

|                                         |                                    |
|-----------------------------------------|------------------------------------|
| POST /api/sos/trigger                   | - Fire an SOS event                |
| GET /sos/dashboard, /sos/alerts         | - SOS monitoring dashboards        |
| POST /sos/volunteer/respond             | - Volunteer responds to a live SOS |
| GET/POST /journey/start,/safe,/cancel   | - Travel-with-me journey lifecycle |
| GET/POST /users/{id}/trusted-contacts   | - Manage trusted contacts          |
| GET/POST /users/{id}/emergency-contacts | - Manage emergency contacts        |

### Health & wellness

|                                              |                           |
|----------------------------------------------|---------------------------|
| GET/POST /doctors/register, /login           | - Doctor onboarding       |
| POST /doctors/book                           | - Book a consultation     |
| GET /doctors/chat/{doctorId}                 | - In-app doctor chat      |
| GET /doctors/video-call/{doctorId}           | - Video consultation room |
| POST /doctors/appointments/{id}/prescription | - Issue a prescription    |
| GET/POST /salons/register, /login            | - Salon onboarding        |
| GET /salons/services/search                  | - Search salon services   |

### Training & self-defense

|                                      |                                      |
|--------------------------------------|--------------------------------------|
| POST /online-class/create            | - Organizer creates a class          |
| POST /online-class/trainee-join/{id} | - Trainee joins a live class         |
| GET /training?                       | - View training status               |
| GET /centres/allacceptedcentres      | - List approved martial arts centres |
| POST /centres/batches/create         | - Create a training batch            |

### Marketplace & commerce

|                                         |                                           |
|-----------------------------------------|-------------------------------------------|
| POST /women-products/cart/add           | - Add product to cart                     |
| POST /women-products/checkout/place     | - Place an order                          |
| POST /women-products/orders/{id}/return | - Request a return                        |
| GET/POST /marketplace/provider/register | - Service provider onboarding             |
| POST /marketplace/classes/enroll        | - Enroll in a marketplace class           |
| POST /payment/create-order, /verify     | - Razorpay order + signature verification |
| GET /users/wallet                       | - View wallet balance                     |

### Community

|                                    |                       |
|------------------------------------|-----------------------|
| POST /stories/create               | - Post a safety story |
| POST /qna/ask, /thread/{id}/answer | - Community Q&A       |

### Admin

|                                                 |                                   |
|-------------------------------------------------|-----------------------------------|
| GET /admin/adminDashboard, /dashboard.meta      | - Platform overview               |
| GET /admin/sos, /sos.json                       | - Live SOS monitoring feed        |
| POST /admin/users/{id}/ban, /verify, /reject    | - User moderation                 |
| POST /admin/doctors/{id}/verify                 | - Doctor verification             |
| POST /admin/sellers/{id}/verify                 | - Marketplace seller verification |
| GET /admin/reports/export-sos, export-incidents | - CSV exports                     |

POST /admin/broadcast/send

- Platform-wide broadcast message

**Note:** almost every module above still runs behind the SecurityConfig gap flagged earlier (blanket permitAll + CSRF disabled) — this is why that fix is Phase 0, not a nice-to-have.

## SECTION 6 — FRONTEND PAGE STRUCTURE (JSP VIEWS)

215 JSP views exist today, server-rendered from WEB-INF/views. Grouped by area:

| Area           | Views (from repo)                                                                                     |
|----------------|-------------------------------------------------------------------------------------------------------|
| Safety core    | panic.jsp, panic-button.jsp, sos-dashboard.jsp, sos-response-page.jsp, journey.jsp, trusted-con       |
| Volunteers     | buddy.jsp, pendingVolunteers.jsp, approvedVolunteers.jsp, reportVolunteer.jsp, rateVolunteer.jsp      |
| Health         | doctor/ (9 views — booking, chat, video call, prescriptions), medical-details.jsp                     |
| Salon & beauty | salon/ (16 views), stylist/ (5 views), stylistServices/ (2 views)                                     |
| Training       | onlineClassManagement.jsp, trainingJourney.jsp, myProgress.jsp, myAttendance.jsp, myBeltPr            |
| Marketplace    | marketplace/ (9 views), women-products/ (9 views), wallet.jsp, payment.jsp, myPayments.jsp            |
| Community      | safety-stories.jsp, questions/ (3 views), reels.jsp, videosPage.jsp, chat.jsp, chat_call.jsp, profile |
| User account   | user/ (18 views), login.jsp, forgotPassword.jsp, resetPassword.jsp, userDashboard.jsp, user-pro       |
| Admin panel    | adminDashboard.jsp, adminSOSMonitor.jsp, adminUserManagement.jsp, adminPendingDoctor                  |
| Shared         | fragments/ (4 shared includes), index/ (6 landing views), livelocation/, safeRoute/                   |

**Not yet present:** no dedicated mobile-optimized views, no PWA manifest, no offline-first shell — confirming the earlier finding that safety-critical features (background SOS, lock-screen alerts) are not achievable in the current browser-only frontend.

## SECTION 7 — DEVELOPMENT ROADMAP

### PHASE 0 — Security & production hardening

Days 1–10

- Re-enable and properly scope CSRF protection (currently disabled project-wide)
- Replace `permitAll()` with real role-based authorization (USER / PROVIDER / ADMIN)
- Authenticate WebSocket handshakes on `/ws-sos` and `/ws-chat`
- Field-level encryption for `medicalDetails` and `trustedContact` data
- Move Razorpay and SMTP secrets out of `application.properties` into environment variables, rotate both

### PHASE 1 — Core safety mobile MVP

Weeks 1–14

- SOS trigger with hardware button / widget / shake gesture
- Background live location during an active SOS session
- Push notifications to trusted contacts (Firebase FCM)
- Trusted contacts UI, battery status, offline SOS fallback
- Convert relevant JSP endpoints to clean JSON responses for the mobile client

### PHASE 2 — Feature parity + quick wins

Weeks 12–20  
(overlaps Phase 1)

- Mobile screens for training videos, class enrollment, wallet, martial arts booking
- Finish and surface Safety Stories, Doctor Consultations, and Q&A — backend already exists, largely completion work

### PHASE 3 — New differentiation features

Weeks 18–28

- Verified gig/earning marketplace tied to the existing trust layer
- Breast cancer / PCOD / menstrual health tracking and reminders
- Referral / Safety Champion earn program, leaderboards, local spotlight

### PHASE 4 — Scale readiness

Post-funding, ongoing

- Redis-backed WebSocket scaling to replace the in-memory STOMP broker
- Flyway-managed migrations, MySQL read replica, S3/R2 for media
- Docker + CI/CD pipeline, Sentry monitoring, load testing

## SECTION 8 — PRODUCTION READINESS & RISK REGISTER

An honest gap list — current state versus what closes each risk — rather than a claim that these protections already exist.

| Risk                                             | Current state                                                  | Fix                                                      |
|--------------------------------------------------|----------------------------------------------------------------|----------------------------------------------------------|
| Every endpoint currently reachable via curl      | SecurityConfig.java has anyRequest().permitAll()               | PermitAll() authorizeHttpRequests() rules per controller |
| CSRF forgery on SOS/payment                      | csrf(AbstractHttpConfigurer::disable) present                  | Disable CSRF, exempt only genuine webhooks               |
| Spoofed SOS/location via WebSockets and /ws-chat | accept unauthenticated connections                             | Feature SSO MFA protections                              |
| Medical & contact data exposed in DB             | is encrypted                                                   | AES-GCM field-level encryption via JPA Attribute         |
| Secrets committed to source                      | Razorpay secret + Gmail app password in application.properties | Move application properties, rotate both credentials     |
| Schema drift across environments                 | hibernate-auto=update auto-migrates on every deploy            | Flyway/Liquibase versioned migrations                    |
| Single point of failure on real-time             | memory SimpleBroker, tied to one JVM                           | Redis Pub/Sub broker once traffic requires multiple JVMs |
| No visibility into failures in production        | no error tracking or monitoring configured                     | Sentry or equivalent APM                                 |
| Data loss on server failure                      | No automated backup strategy visible                           | Scheduled MySQL backups to S3/R2 with retention          |

## SECTION 9 — THIRD-PARTY SERVICES & API KEYS

| Service                  | Used for                                                 | Cost model                  | Status  | Note                                                                           |
|--------------------------|----------------------------------------------------------|-----------------------------|---------|--------------------------------------------------------------------------------|
| Razorpay                 | Payment processing                                       | Transaction fee per payment | LIVE    | Test-mode keys currently in source — rotate before production use              |
| Gmail SMTP               | Trusted-contact email alerts, Pass (single mailbox)      | Free (single mailbox)       | LIVE    | Not a dedicated transactional provider — deliverability risk at scale          |
| Google Maps API          | Referenced in config for location features               | Pay per use                 | LIVE    | Placeholder key in application.properties — needs a real key issued and scoped |
| Firebase Cloud Messaging | Push notifications                                       | Free tier generous          | PLANNED | Required for reliable, fast SOS delivery once the mobile app ships             |
| AWS S3 / Cloudfront      | Media storage (photos, videos, \$0.015/GB) no egress fee | \$0.015/GB no egress fee    | PLANNED | Move storage off the app server, enables CDN delivery                          |
| Sentry                   | Error tracking & monitoring                              | Free tier for small apps    | PLANNED | No monitoring currently exists in production                                   |